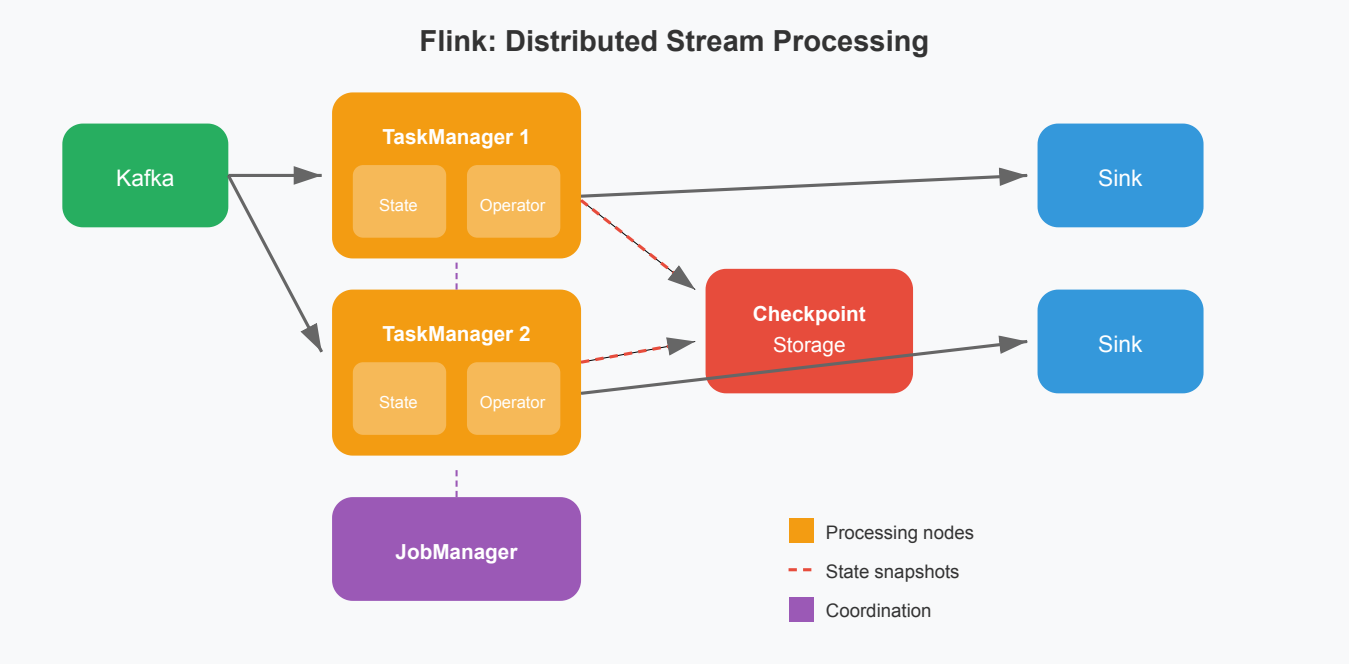
Modern Stream Processing: Flink and Kafka Streams

systemdesignschool.io/fundamentals/modern-stream-flink



Modern Stream: Flink & Kafka Streams

The previous articles covered stream processing concepts: windowing, event time, and delivery guarantees. Implementing these patterns with raw Kafka consumers is possible but requires substantial work. You'd build your own state management, handle consumer rebalancing, and coordinate exactly-once semantics manually.

Modern stream processing frameworks handle this complexity for you. Apache Flink and Kafka Streams are two widely adopted options, each with a different deployment model.

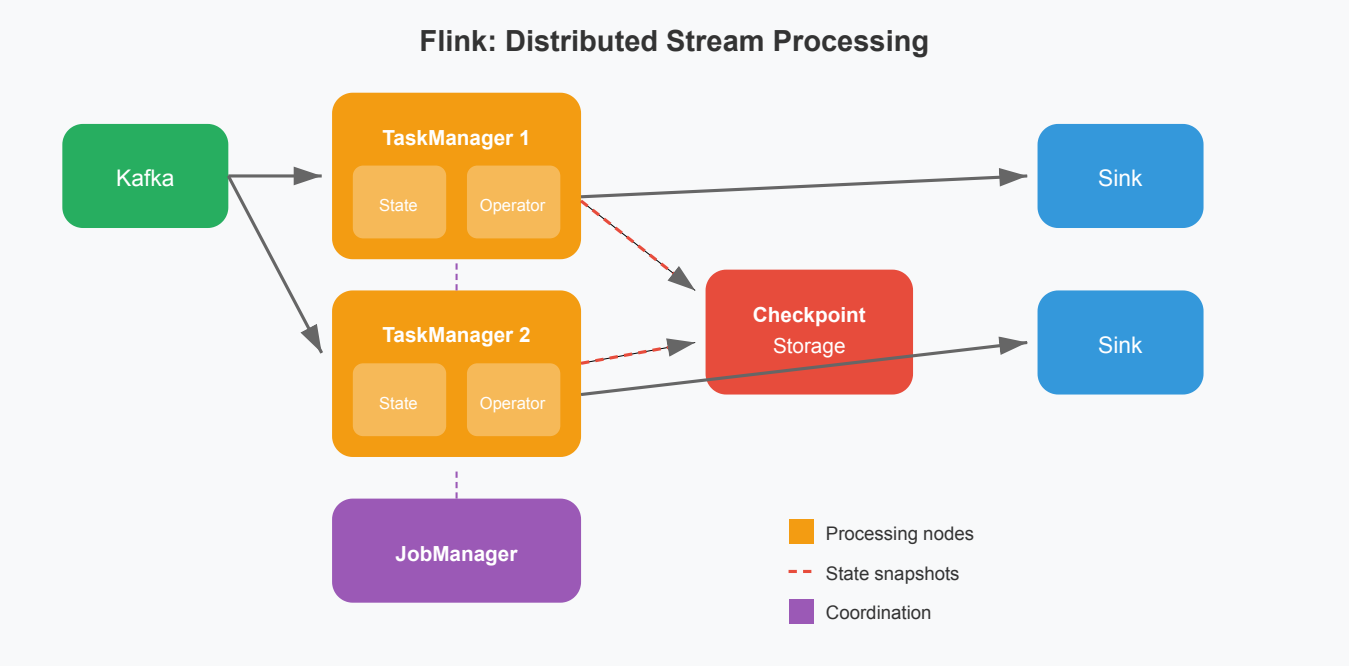
What These Frameworks Provide

Both Flink and Kafka Streams give you:

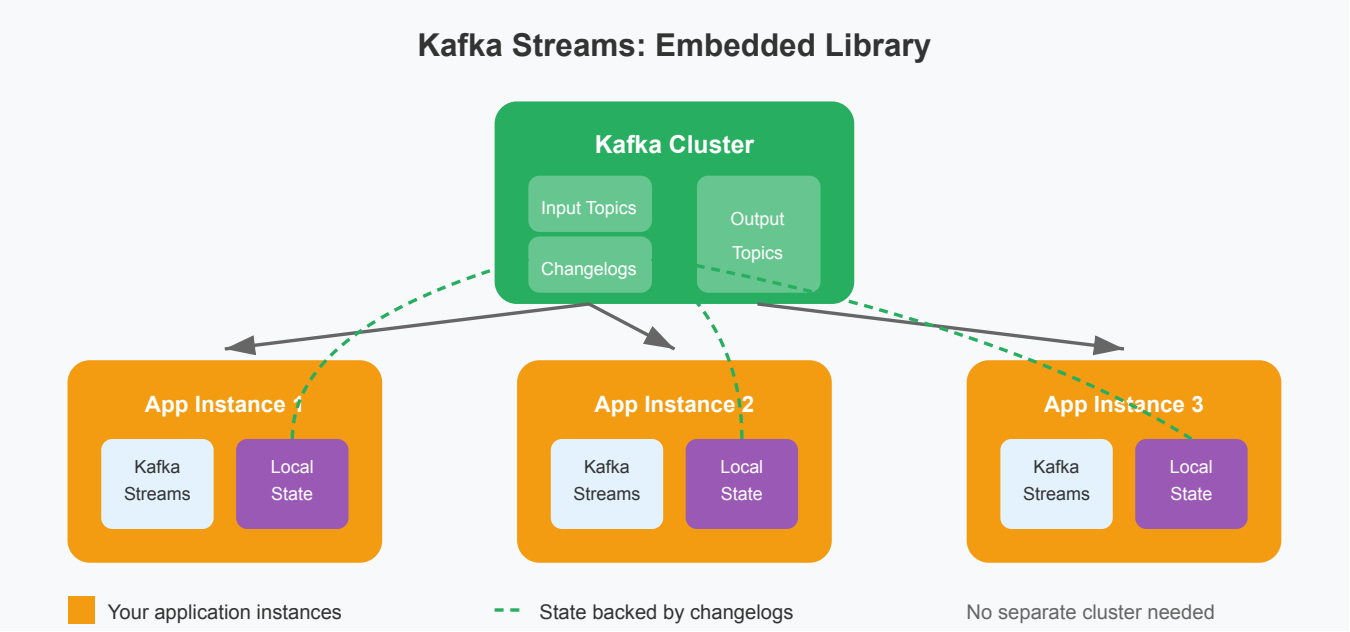
- **Managed state.** The framework tracks per-key state (running counts, window contents) and handles redistribution when workers scale up or down.
- **Fault tolerance.** Automatic checkpointing and recovery. When a worker fails, processing resumes from the last checkpoint without data loss.
- **Windowing and event time.** Built-in support for tumbling, sliding, and session windows with watermarks for handling late-arriving events.
- **Exactly-once semantics.** Coordinated commits ensure events aren't lost or double-counted, without you building that coordination yourself.

Flink vs Kafka Streams: Deployment Model

The key difference is how you run them.



Apache Flink is a distributed cluster. You submit jobs to a JobManager, which schedules work across TaskManager workers. Flink handles diverse sources—Kafka, databases via CDC, files, and more. The trade-off is operational overhead: you run and monitor a separate cluster.



Kafka Streams is a library embedded in your application. Your application instances *are* the processing cluster. Scale by running more instances. Kafka handles partition assignment automatically. The trade-off is that it's Kafka-centric—your inputs and outputs should be Kafka topics.

When to Use Which

Choose Flink when:

- Sources include databases, files, or non-Kafka systems
- You need very low latency (sub-second)
- You're already operating distributed infrastructure

Choose Kafka Streams when:

- Kafka is already your event backbone
- You want to avoid managing a separate cluster
- Simpler operational model matters more than connector flexibility

For system design interviews, what matters is understanding *why* these frameworks exist: they abstract away the hard parts of stateful stream processing (state management, fault tolerance, exactly-once) so you can focus on business logic.

What's Next

Modern stream processors handle stateful, fault-tolerant processing. But many systems need both real-time updates and batch processing for historical completeness. The next section explores **hybrid architectures**—Lambda and Kappa patterns that combine both approaches.